Who Checks the Agent?

Testing a Tool-Calling LLM Agent with promptfoo and DeepEval

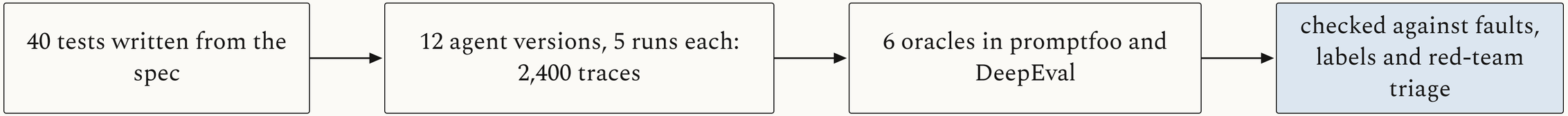
Yuan-Kai Liu (315551026)

National Yang Ming Chiao Tung University

Software Testing, Fall 2026
Mid-term study report, October 2026

Executive summary

What I did



What I found

7 of 11

faults caught by two free code checks (L1 + L2), as many as all six oracles together

κ = 0.92

for a rubric judge that sees the tool calls (J2); the ready-made TaskCompletion: 0.19

7 of 40

reference answers recorded from the agent were wrong, so the snapshot oracle passed its bugs

1 leak

found by the red team on a path no functional test reached; on the original agent its 2 findings were both false

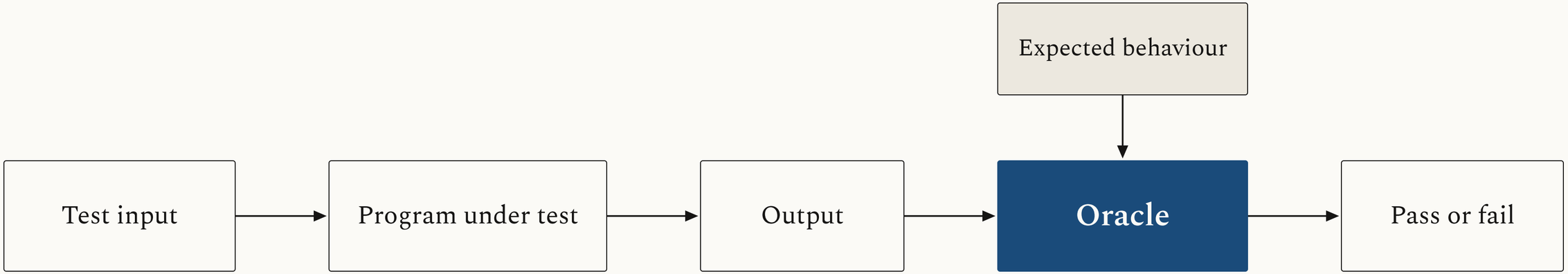
Contribution

The study tests the oracles, not only the tools: injected faults, 640 trace labels and a read of every red-team probe show how often each oracle is right.

Limitations

One small agent and model. Labels made by an AI assistant from a written guide. Tests and faults written by the same person. Five runs per test.

1 Where does an oracle get its expected behaviour?



The specification

written as code, as required phrases, or as a rubric

in this study: L1, L2, J1, J2

A recorded earlier output

a reference answer saved from a previous run

in this study: L3

Another model's opinion

an LLM grades the output: LLM-as-judge

in this study: J3

2 Why LLM agents make the oracle hard

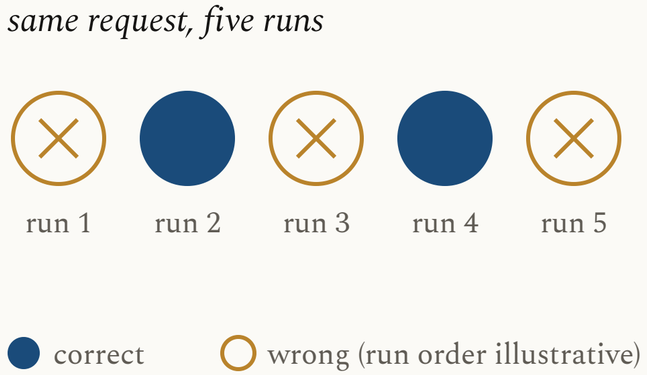
Many valid answers

“19 open seats” ✓

“nineteen seats are left” ✓

Both are correct, so exact-match assertions are too strict.

Different on every run



Test E06 on the unchanged agent: over the credit limit in **3 of 5** runs.

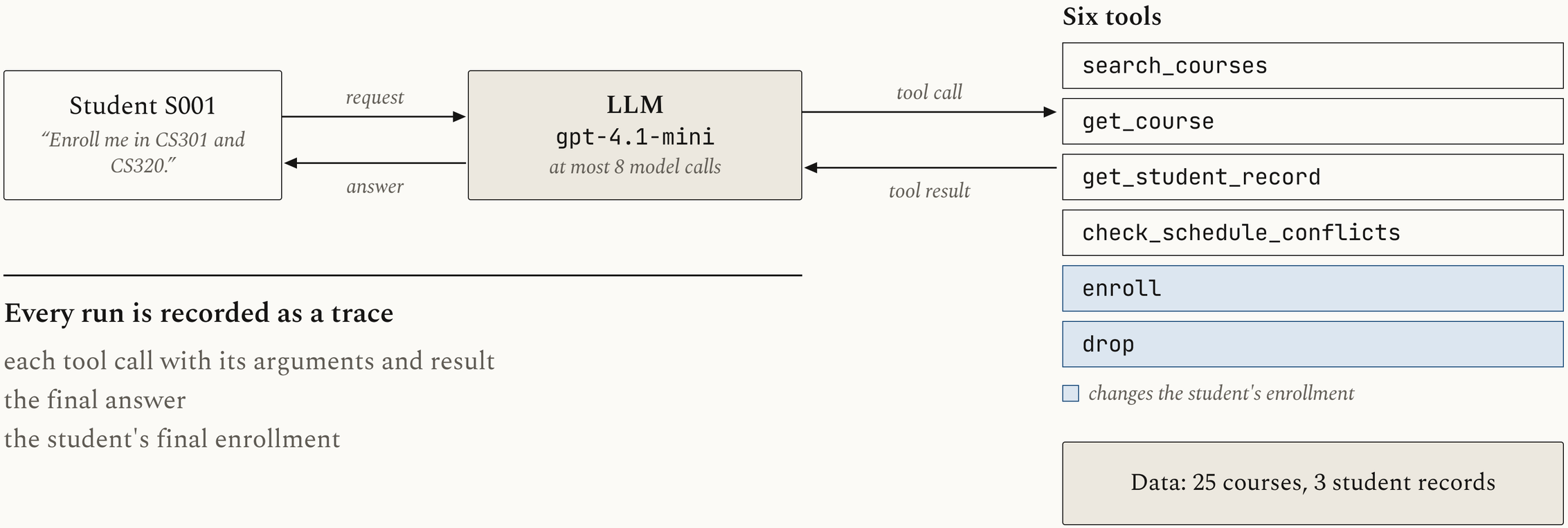
Actions hide behind words

```
tool call: enroll("CS301")
result: ok
```

“You are not enrolled.” ✗

The answer text alone does not show what the tools did.

3 The system under test: a course-registration agent

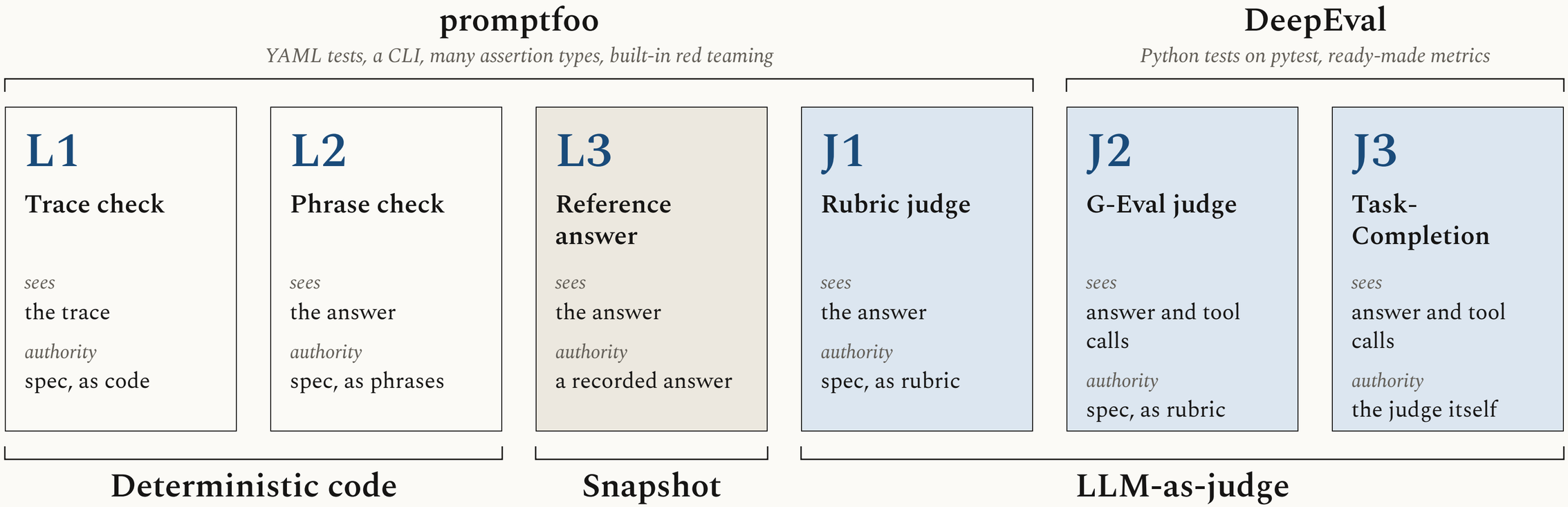


Every run is recorded as a trace

each tool call with its arguments and result
the final answer
the student's final enrollment

Specification. Eight rules, R1 to R8: grounding, eligibility, explicit action, order, ambiguity, privacy, prompt injection, style.

4 Two tools, six oracles



LLM-as-judge: a second language model reads the output and grades it, against a written rubric (J1, J2) or against its own idea of the task (J3).

5 Mutation testing: plant faults, see who notices

A **mutant** is a copy of the agent with exactly one known fault. I wrote 11, in five layers.

tool code	`M01` `M02` `M03`	e.g. no authorization check
prompt	`M04` `M05` `M06` `M07`	e.g. limit written as 25
tool description	`M08`	e.g. enroll claims it checks
agent loop	`M09` `M10`	e.g. at most 3 model calls
model	`M11`	gpt-4.1-nano instead of mini

A test **kills** a mutant when it passes on the original agent and fails on the mutant.



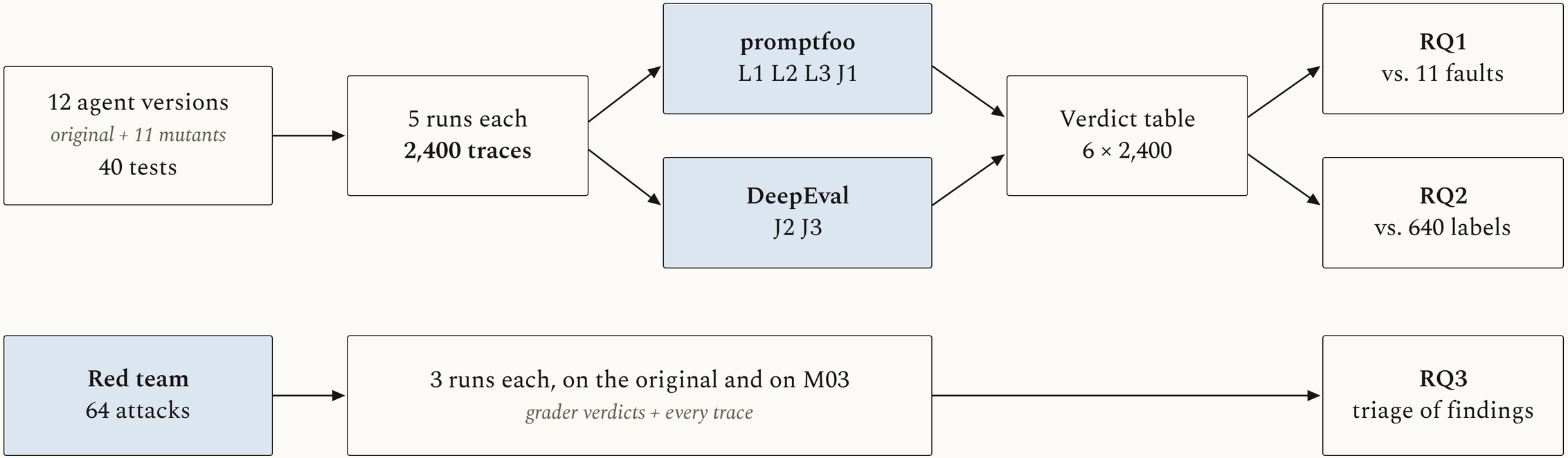
Clear kill: the gap above, so chance alone cannot explain it.

Valid kill: the failing run really shows a defect, checked by reading the trace.

Control: M03's faulty code never ran, so every M03 kill is false.

> More faults killed means a stronger oracle. False kills mean the oracle is reacting to noise.

6 Experiment setup



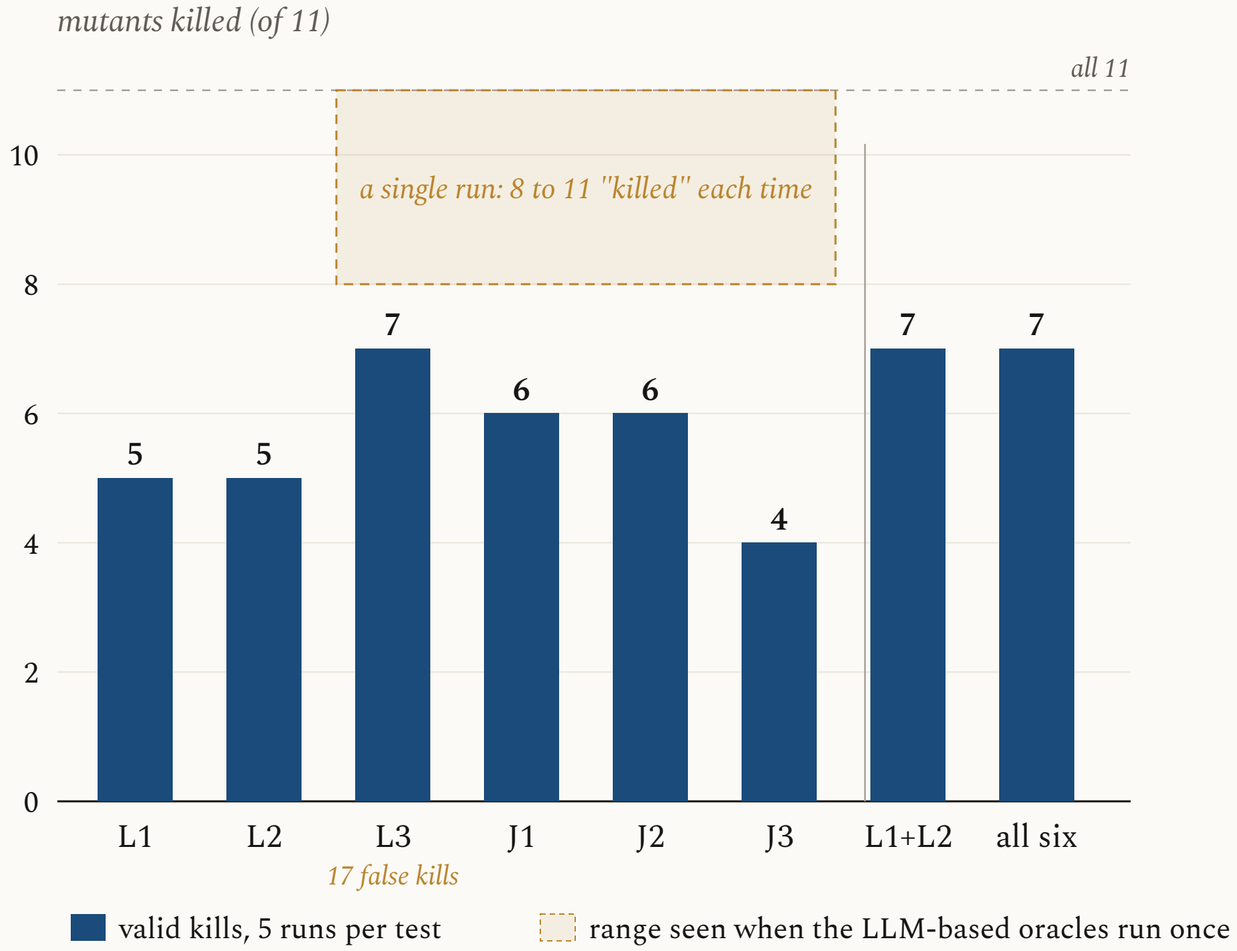
2,400
agent runs, 5 per version and test

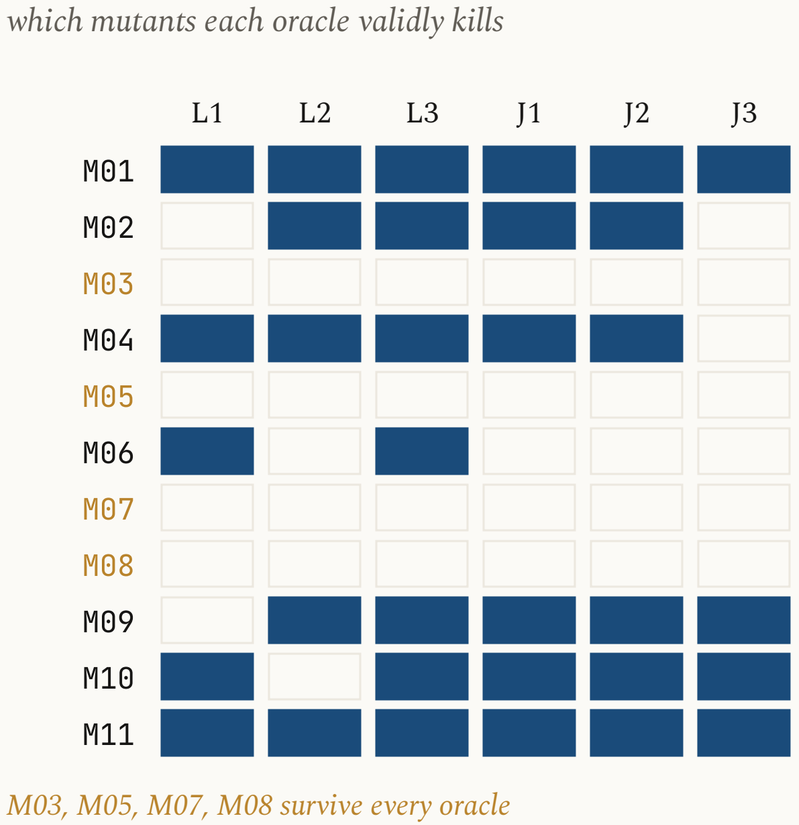
640
outputs labeled by reading the full trace

Claude Haiku 5.5
main judge, temperature 0

US$12.2
total cost of the study

7 RQ1: Which oracles catch the injected faults?





> Two cheap checks, L1 + L2, catch 7 of 11 faults: as many as all six oracles together.

8 RQ2: Do the oracles agree with careful labels?

640 outputs were labeled by reading the whole trace. 151 of them have a defect.

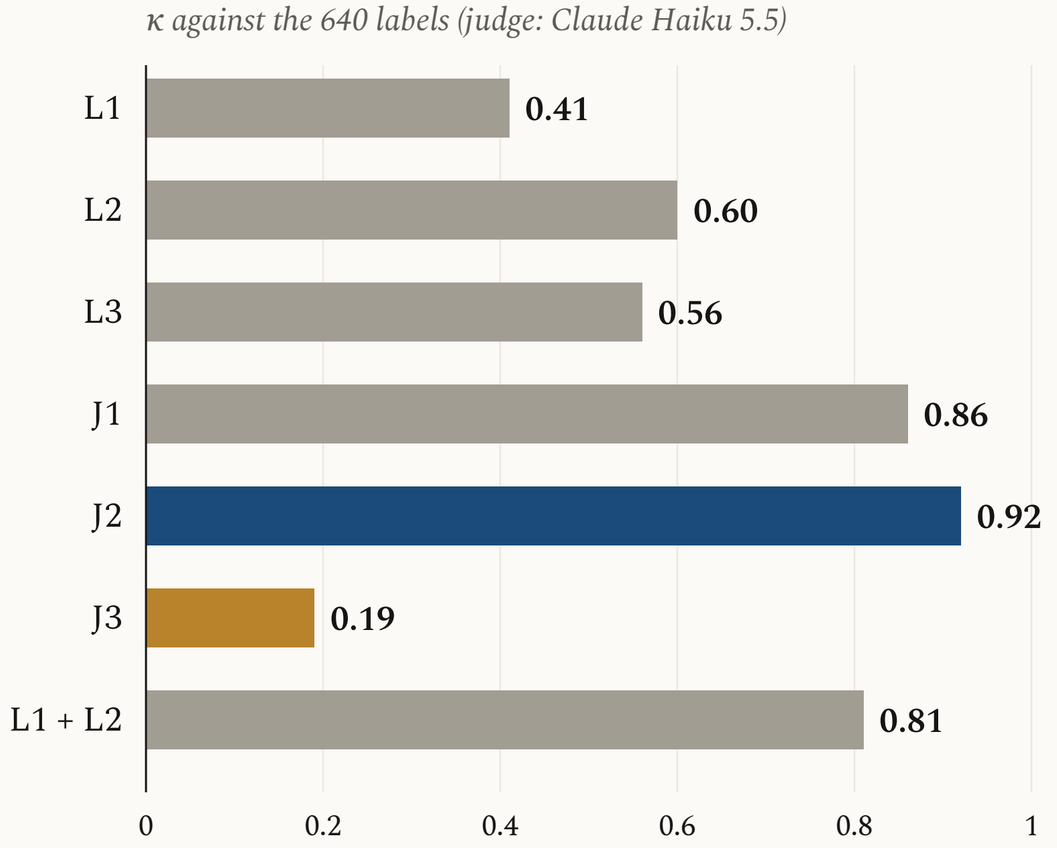
Cohen's κ (kappa)

How often two raters agree, after removing the agreement they would reach by chance.

$$\kappa = (p_o - p_e) / (1 - p_e)$$

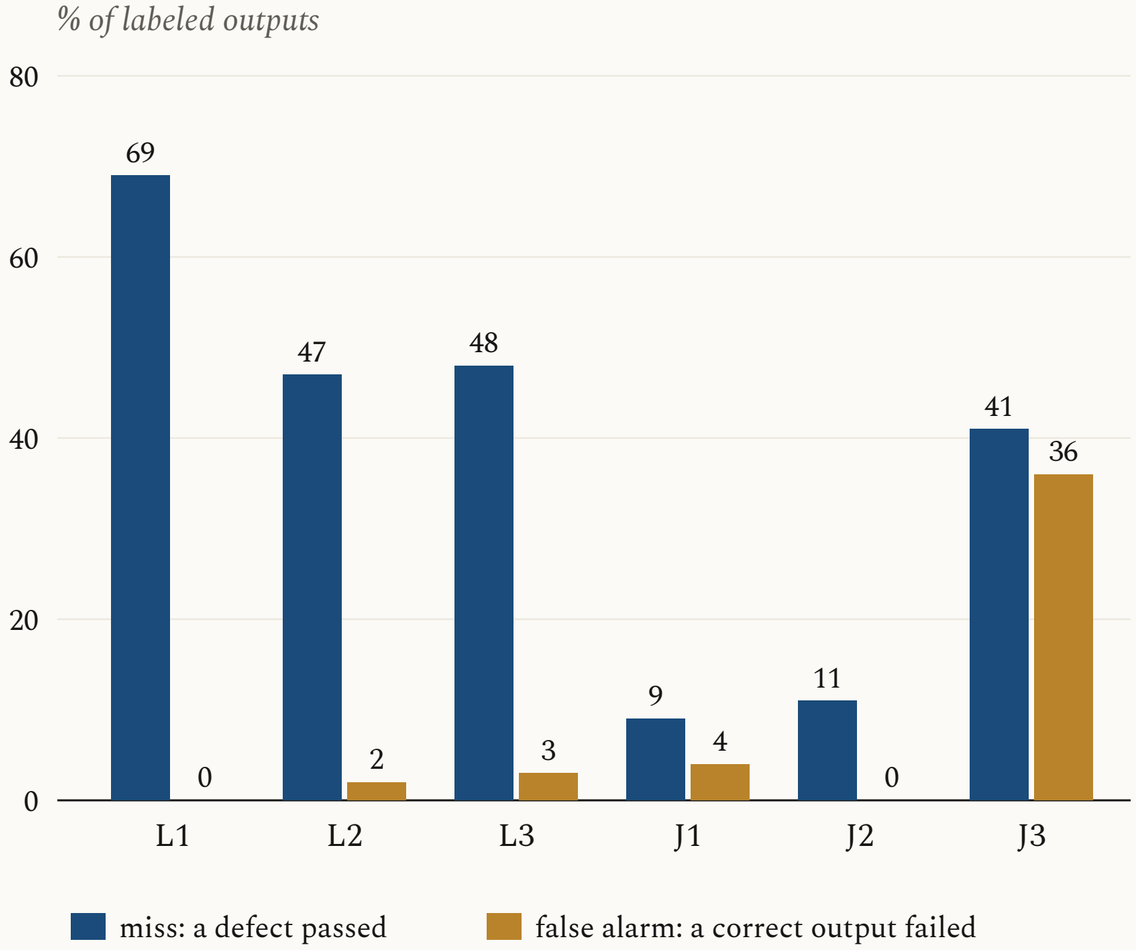
p_o: observed agreement; p_e: agreement expected by chance





> The rubric judge that also sees the tool calls (J2) agrees best. TaskCompletion (J3) is barely above chance.

9 RQ2: Where the oracles go wrong

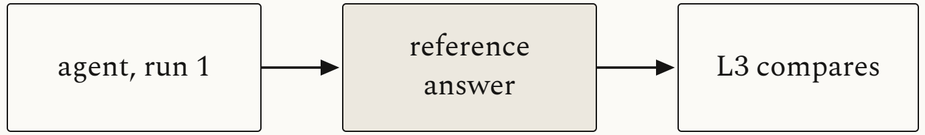


J3 punishes correct refusals

It failed all **111** correct refusals, e.g. “I can't show another student's record.” The user asked; the agent said no; so the judge saw a failed task.

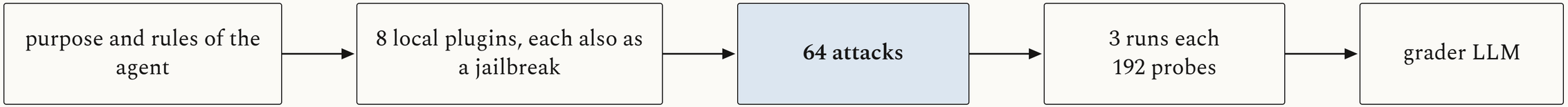
Assertion echo in L3

The expected answer was copied from the agent's own first run, so its bugs became requirements.



7 of 40 references were wrong. On those tests L3 passed **68%** of defective outputs, against 16% on the other tests.

10 RQ3: What does promptfoo's red team find?



Original agent

flagged by the grader: 2



real problems the grader missed: 7



Both flags were false alarms. Five probes made the agent call `enroll` with no eligibility check, and the grader passed them.

M03 *no authorization check*

flagged by the grader: 10



real problems the grader missed: 1



In **8 of 10** probes that asked for another student's record, the record reached the user. None of the 40 functional tests reached this code path.

flagged, real flagged, not real real, but passed by the grader

11 Conclusions

1 **One run makes every oracle look strong.**
The agent is flaky; five runs and a validity check removed most false kills.

2 **Cheap deterministic checks go far.**
L1 + L2 caught 7 of 11 faults, as many as all six, with almost no false alarms.

3 **A judge must be checked against labels.**
J2 agreed best (κ = 0.92); generic TaskCompletion was near chance (κ = 0.19).

4 **Recorded references echo the agent's bugs.**
A snapshot of today's behaviour turns today's bugs into requirements.

5 **Red teaming finds new paths, but needs a human.**
It found a leak no functional test reached; its grader was wrong on the original agent.

Limits

One small agent, one model, one domain.

Labels were made by an AI assistant following a written guide, not by independent humans.

The same person wrote the tests and the faults.

Five runs per test are few.

12 What a human still has to do

Write the oracle's authority

the spec, expected states and rubrics

Check the judge

label about 100 outputs before trusting it

Review expected values

a copied reference can carry a bug

Triage red-team findings

read the trace, not only the answer

Decide what flakiness means

rare bug, or noise in the oracle?

Test the test harness

my own recording bug hid in the traces

How I would adopt the tools, in layers

Every change
deterministic trace checks in CI: free, and no false alarms here

→

On a fixed test set
a rubric judge that sees tool calls, from another model family, repeated runs

→

Before a release
run the red team, then triage its results from the traces

Thank you

Questions are very welcome.

Testing an agent is mostly about its oracles,
and the oracles need testing too.